Running guide

Metal hydride ML project · Avinash (230151005) · https://github.com/avinas29/Hydrogen-Storage-Project

How to install and run the project on your own computer: the interactive app first, then (optionally) the analysis pipeline. About 15 minutes, most of it waiting for the install.

Tested on macOS with Python 3.13. The Windows commands below are the standard equivalents.

What you need

Requirement	Details
Python	**3.12 or 3.13** (3.13 recommended; the pinned NumPy/SciPy need at least 3.12). Check with `python3 --version` (macOS/Linux) or `py --version` (Windows). Download from https://www.python.org/downloads/ if needed. On Windows, tick "Add python.exe to PATH" during installation.
Git	optional. Without Git, download the ZIP from GitHub instead (step 1).
Disk / network	about 1.5 GB for the Python packages; an internet connection for the install

1. Get the code

With Git:

```
git clone https://github.com/avinas29/Hydrogen-Storage-Project.git
cd Hydrogen-Storage-Project
```

Without Git: open https://github.com/avinas29/Hydrogen-Storage-Project, click **Code → Download ZIP**, unzip it, and open a terminal (macOS: Terminal; Windows: PowerShell or Command Prompt) in the unzipped folder.

2. Create an environment and install the packages

macOS / Linux

```
python3 -m venv .venv
.venv/bin/python -m pip install --upgrade pip
.venv/bin/python -m pip install -r requirements.txt
```

Windows

```
py -3.13 -m venv .venv
.venv\Scripts\python -m pip install --upgrade pip
.venv\Scripts\python -m pip install -r requirements.txt
```

The install takes 5–10 minutes. The commands call the environment's Python directly, so there is no need to "activate" it. If `py -3.13` is not found, use `py -3.12` or `python`.

3. Check the installation (optional, 10 seconds)

macOS / Linux:

```
.venv/bin/python -m pytest -q tests
```

Windows:

```
.venv\Scripts\python -m pytest -q tests
```

Expected result: **71 passed, 6 skipped**. The 6 skipped tests check the parser of the raw source files, which are not included in the repository (see `data/raw/README.md`). Everything else, including a headless run of every app page, passes.

4. Start the app

macOS / Linux:

```
.venv/bin/python -m streamlit run app.py
```

Windows:

```
.venv\Scripts\python -m streamlit run app.py
```

A browser tab opens at **http://localhost:8501** (open it by hand if it does not). Stop the app with **Ctrl + C** in the terminal. The first start takes a few seconds while the model loads.

5. What to try

Page (left sidebar)	Try this	What you should see
Overview	–	725 compositions, cross-validated MAE 5.26 kJ/mol, read from the result files
Property Predictor	`LaNi4.2Al0.8`	ΔH ≈ 38.7 kJ/mol, 80% interval 34.3–41.6, inside the applicability domain, nearest known alloy LaNi4.25Al0.75 (38.3)
Property Predictor	`Ti0.5V0.5`	flagged **outside the applicability domain** (nearest training alloy at L1 = 0.33; typical error ≈ 13 kJ/mol)
Property Predictor	`LaNi5`	warning that LaNi5 is in the training data, so the measured value (32.15 kJ/mol) is the one to quote
Application Requirement Designer	choose "Mobile hydrogen storage", go through the steps and run	rejected materials with the requirement each one fails, a ranked list with evidence classes, a capacity–heat Pareto front, CSV/JSON export
Candidate Discovery Lab	choose an alloy system (e.g. Ti-Fe-Mn) and run	generated compositions, rejected ones with reasons, ranked candidates (ΔH predicted, ΔS assumed)
Model Performance	–	cross-validation, hold-out and leave-one-family-out results, applicability-domain calibration

6. Reproduce the analysis (optional)

All results, figures and the saved model are already in the repository, so this is only needed to regenerate them. These steps run from the included processed data. Commands are shown for macOS/Linux; on Windows, replace `.venv/bin/python` with `.venv\Scripts\python` and leave out the `MPLCONFIGDIR=.mplcache` prefix.

```
.venv/bin/python -m src.features.build
.venv/bin/python -m src.models.train
.venv/bin/python -m src.models.report
MPLCONFIGDIR=.mplcache .venv/bin/python -m src.visualization.ml_plots
.venv/bin/python -m src.thermodynamics.analysis
.venv/bin/python -m src.thermodynamics.pci_kinetics
.venv/bin/python -m src.screening.screen
.venv/bin/python -m src.app.demo
```

In order: composition descriptors; model training with nested grouped cross-validation (the slow step, about 6 minutes); the ML report and figures; Van't Hoff tables; PCI plateaus and kinetics; mobile/stationary screening; the two demonstration scenarios of the platform. Retraining reproduces the saved metrics (checked on a fresh clone: differences only in the last floating-point digit, about 10^{-15}).

The full list of phase commands is in `README.md`. Phase 1 (`src.data.run_audit`) and the raw-parser tests need the original source files; `data/raw/README.md` explains where to download them (the Purdue toolbox from GitHub and ML-HydPARK from Zenodo).

Rebuilding the report (`python -m report.build_report`) also needs LibreOffice, and the viva guide needs Node.js and Google Chrome. Neither is required to run the app.

Troubleshooting

Problem	Fix
`python3` / `py` not found	install Python 3.13 from python.org (Windows: tick "Add python.exe to PATH")
pip starts compiling packages for a long time, or fails with a compiler error	the Python version is too new or too old for the pinned packages; use Python 3.13 (or 3.12)
`ModuleNotFoundError` when starting the app	the command used a different Python; run it as `.venv/bin/python -m streamlit ...` (Windows: `.venv\Scripts\python -m streamlit ...`) from the project folder
`InconsistentVersionWarning` from scikit-learn when the model loads	a different scikit-learn version is installed; reinstall with `pip install -r requirements.txt` (the saved model was created with scikit-learn 1.9.1)
"Port 8501 is already in use"	`.venv/bin/python -m streamlit run app.py --server.port 8502`, then open http://localhost:8502
Windows PowerShell blocks activation scripts	no activation is needed; use the full `.venv\Scripts\python` commands above
browser does not open	open http://localhost:8501 manually